

Lab 5

2. Drawing Shapes in Java

2D Graphics in Java

Dr. Andreas S. Maniatis
Assistant Professor

School of Computer Science
COMP.2800 – Software Development [2026W]

Tuesday, February 10th, 2026 | ER 3120



University of Windsor

Part I

Setting up the Environment





JDIALOG



University of Windsor

JDialog

JDialog object is very similar to JFrame.

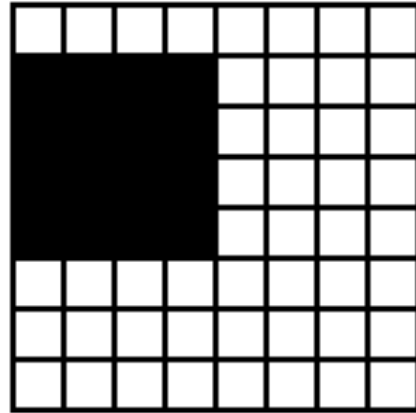
The main differences are:

- **Default Modality:** JFrame is not modal by default. Modal means that execution is paused until the component is disposed. Non-modal components, on the other hand, do not pause execution.
- **Use Case:** JFrame is commonly used as the main window for an application, while JDialog is used for pop-up dialogs or secondary windows that need to interact with the user.

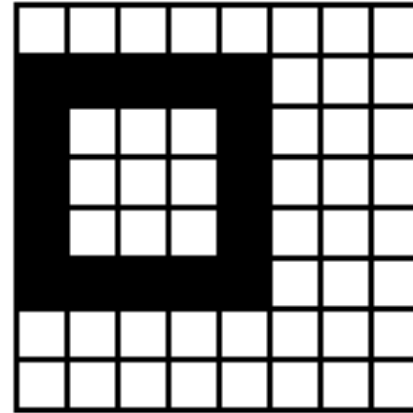


Draw vs Fill

In java.awt.Graphics context, any function which has “draw” in its name means that its going to draw the shape with an outline. Any function with “fill” in its name means that its going to fill the shape.



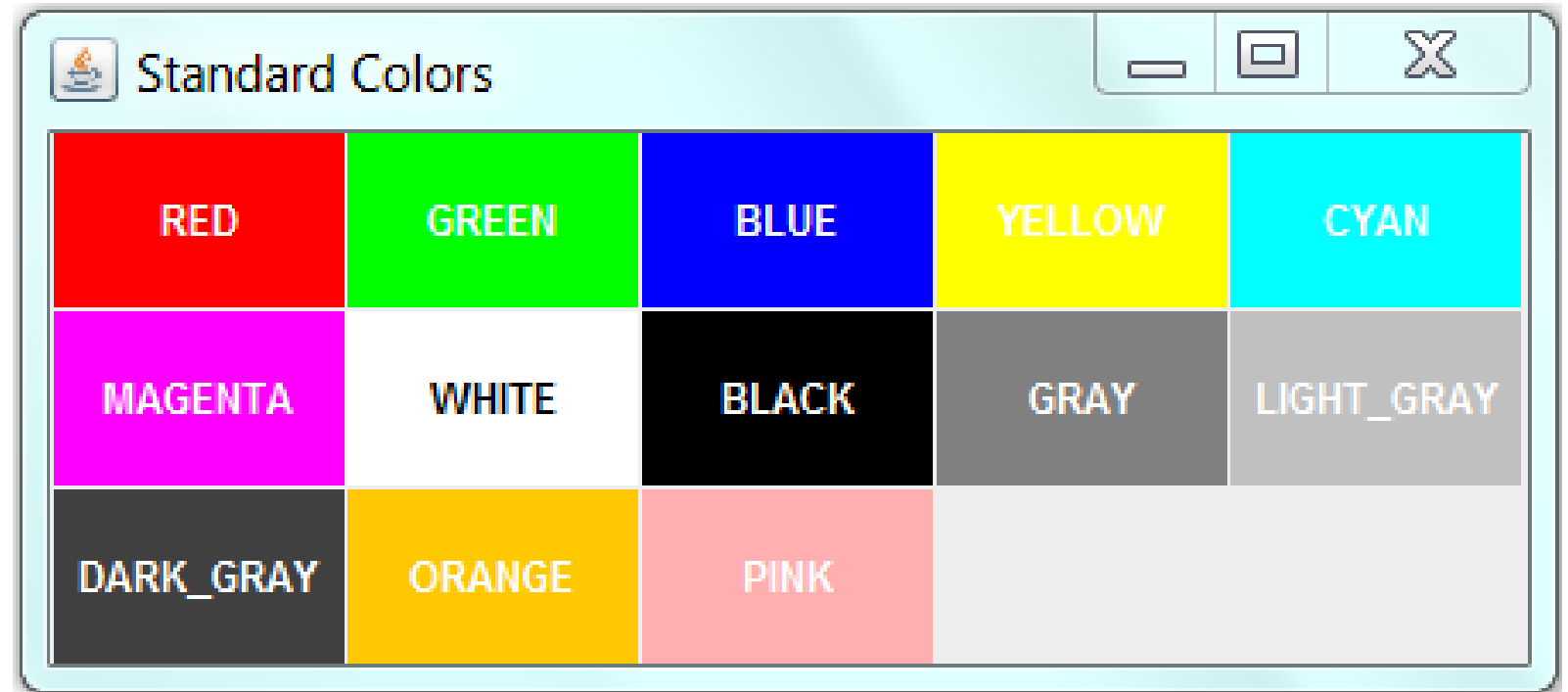
```
g.fillRect(0,1,4,4);
```



```
g.drawRect(0,1,4,4);
```

Java color constants:

Color.BLACK
Color.BLUE
Color.CYAN
Color.DARK_GRAY
Color.GRAY
Color.GREEN
Color.LIGHT_GRAY
Color.MAGENTA
Color.ORANGE
Color.PINK
Color.RED
Color.WHITE
Color.YELLOW



● JPanel vs Canvas

○ For drawing graphics exclusively, we use a Canvas instead of a JPanel. Canvas is similar but more flexible and supports custom buffering techniques.

Creating the Canvas

- We create a `GameCanvas` class which extends the `java.awt.Canvas` class. Creating a separate class for drawing makes it easier to manage code.
- We want the drawing to be done on a different thread. That's why we implement `Runnable` which means it will have a `run()` method. We create a `start()` function which will start our game thread. `thread.start()` will invoke the `run()` method since our class is attached to the thread.



Creating the Canvas

- We can create a canvas by extending the `Canvas` class and implementing the `Runnable` interface. This allows us to create a canvas that can be run on a separate thread.
- We will also create a `Thread` object and start it, so that the canvas can be updated continuously.

```
import java.awt.*;

public class GameCanvas extends Canvas implements Runnable {

    private Thread thread;

    public void start() {
        thread = new Thread(this, "Game Thread");
        thread.start();
    }

    @Override
    public void run() {
        // everything in here is on the Game Thread
    }

}
```



● Creating the `Main` Class

- • We create the main window. We instantiate our `GameCanvas` class and add it to the main window.
- The dimensions of the `Canvas` are adjusted to the window size automatically.
- We are making the window non-resizable.

Creating the Main Class

- We create a class

- The class is automatically

- We add

```
import javax.swing.*;

public class Main {

    public static void main(String[] args) {
        JFrame mainWindow = new JFrame("Graphics Example");
        mainWindow.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        mainWindow.setSize(960, 540);
        mainWindow.setLocationRelativeTo(null);
        mainWindow.setResizable(false);

        GameCanvas gameCanvas = new GameCanvas();
        mainWindow.add(gameCanvas);
        mainWindow.setVisible(true);

        gameCanvas.start();
    }
}
```

What happens after we execute

- If we execute the program right now, there will be 3 important threads executing in parallel. For simplicity, think of a thread as a sub-program running in parallel with our main program.
- Typically, there are a lot more than 3 threads, but the ones that we should know about are the following:
 - Main Thread (this thread is within every java program. Can be thought of as the parent of all threads)
 - Event Dispatch Thread (these are from swing components like JFrame)
 - Game Thread (this is the thread we created)
- Our Game Thread will be running parallel to the other threads, which will provide a smoother experience.



- Creating a `tick()` and `render()` function

```
private void tick() {  
    // update the game (e.g player movement, enemy movement)  
}  
  
private void render() {  
    // draw the game (e.g draw the player, draw the world, draw the enemy)  
}
```

● Setting up a simple game loop

- • This game loop is not very efficient as it's going to update and draw our game as fast as possible. This could be an issue because the speed of the animations will be inconsistent because they are solely dependent on the CPU speed which can vary based on the type of processor and even within the process itself.
- Ideally, we would want to update (tick) the game only a certain number of times per second. For example, 60 times per second. But we want to draw the game (render) at maximum capacity (FPS).
- We will change this later.

-

```
@Override
public void run() {
    while(true) {
        tick();
        render();
    }
}
```

Rendering

- We create a buffer strategy with 2 buffers to use double buffering (no need to focus on this too much).
- In the `render()` method, we obtain the graphics associated with the buffer and can use it to draw anything.



Rendering

- We create a buffer strategy with buffering (no need to focus on the details)
- In the render() method, we obtain a Graphics2D object from the buffer strategy with the buffer and can use it to draw anything

```
private Thread thread;  
private BufferStrategy bufferStrategy;  
  
public void start() {  
    this.createBufferStrategy(2);  
    bufferStrategy = this.getBufferStrategy();  
  
    thread = new Thread(this, "Game Thread");  
    thread.start();  
}
```

```
private void render() {  
    Graphics2D g2d = (Graphics2D) bufferStrategy.getDrawGraphics();  
  
    // draw graphics here  
  
    g2d.dispose();  
    bufferStrategy.show();  
}
```

Drawing a black background

- We can simply draw a black rectangle that is the size of the canvas. This will give an illusion of a black background.

```
private void render() {  
    Graphics2D g2d = (Graphics2D) bufferStrategy.getDrawGraphics();  
  
    g2d.setColor(Color.black);  
    g2d.fillRect(0, 0, this.getWidth(), this.getHeight());  
  
    g2d.dispose();  
    bufferStrategy.show();  
}
```

Output



Drawing a blue square

```
private void render() {  
    Graphics2D g2d = (Graphics2D) bufferStrategy.getDrawGraphics();  
  
    g2d.setColor(Color.black);  
    g2d.fillRect(0, 0, this.getWidth(), this.getHeight());  
  
    g2d.setColor(Color.blue);  
    g2d.fillRect(100, 100, 100, 100);  
  
    g2d.dispose();  
    bufferStrategy.show();  
}
```

Result



Rotating graphics

- We can rotate any type of graphics (rectangles, images, text, ovals, etc.).

```
private void render() {
    Graphics2D g2d = (Graphics2D) bufferStrategy.getDrawGraphics();
    // Drawing Begins_____

    g2d.setColor(Color.black);
    g2d.fillRect(0, 0, this.getWidth(), this.getHeight());

    g2d.setColor(Color.blue);

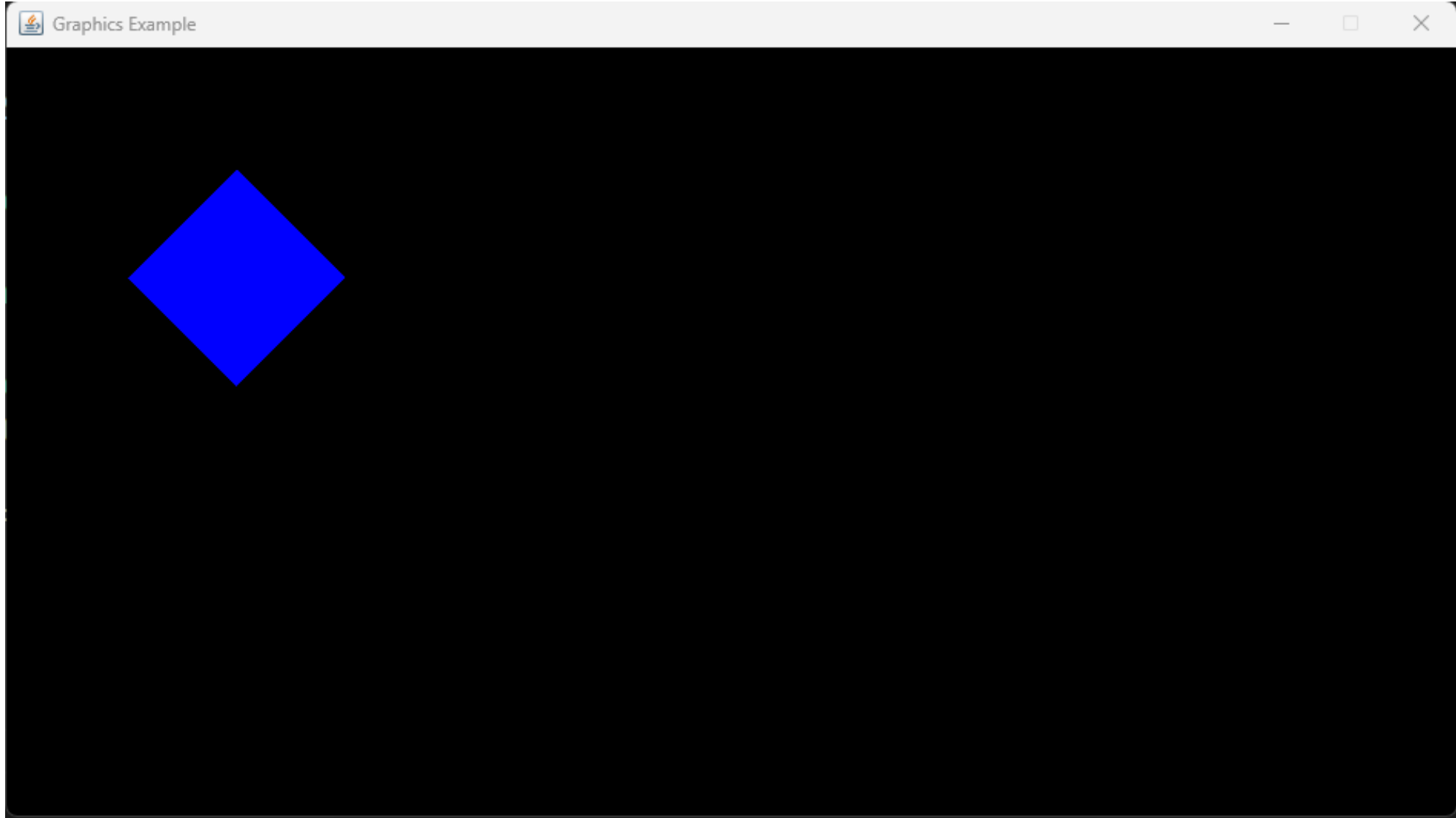
    // save the current state of the graphics to this object
    AffineTransform oldTransform = g2d.getTransform();

    g2d.rotate(Math.toRadians(45), 150, 150); // (150, 150) sets the point of rotation
    g2d.fillRect(100, 100, 100, 100); // draw the rectangle at (100, 100) position

    // reset the transformations back to normal, so it doesnt rotate our images
    g2d.setTransform(oldTransform);

    // Drawing Ends_____
    g2d.dispose();
    bufferStrategy.show();
}
```

Output



Drawing Text (Part 1)

```
g.setColor(Color.black);  
g.fillRect(0, 0, this.getWidth(), this.getHeight());  
  
g.setColor(Color.yellow);  
g.drawString("This is my drawn string!!", 100, 100);
```



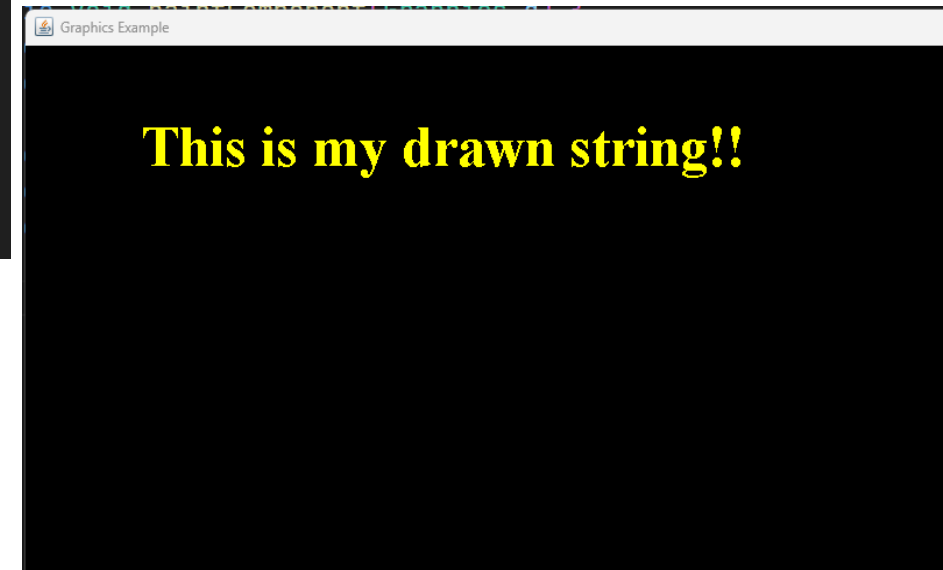
Graphics Example

This is my drawn string!!

Drawing Text (Part 2)

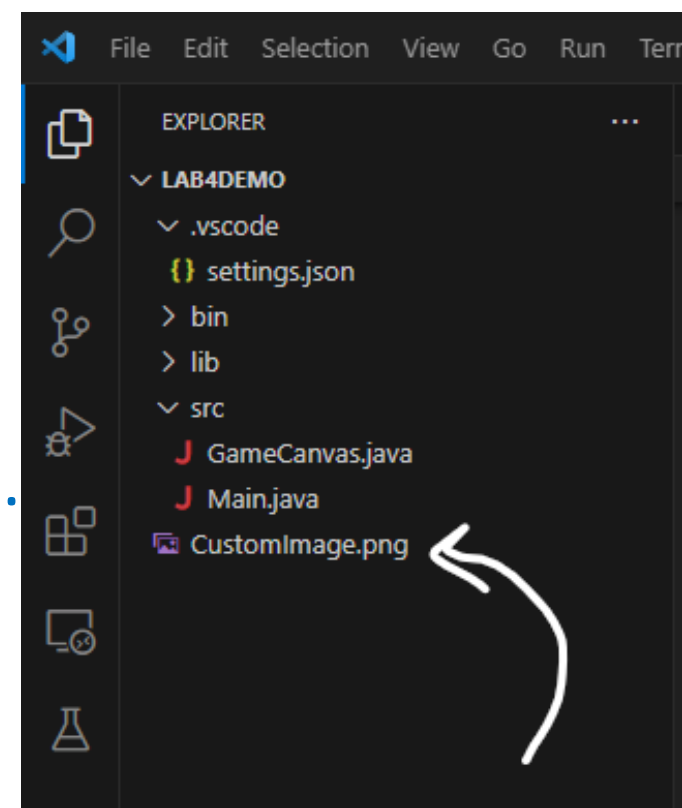
- We can change the font to any font.
- We can also specify the font size and font type.

```
g.setColor(Color.black);  
g.fillRect(0, 0, this.getWidth(), this.getHeight());  
  
g.setColor(Color.yellow);  
g.setFont(new Font("Times New Roman", Font.BOLD, 48));  
g.drawString("This is my drawn string!!", 100, 100);
```



Drawing images (Part 1)

- First, we need to load an image. In Java graphics an image is represented by the `BufferedImage` class.
- The file path is relative to the working directory. In this case it's the Lab4Demo folder.



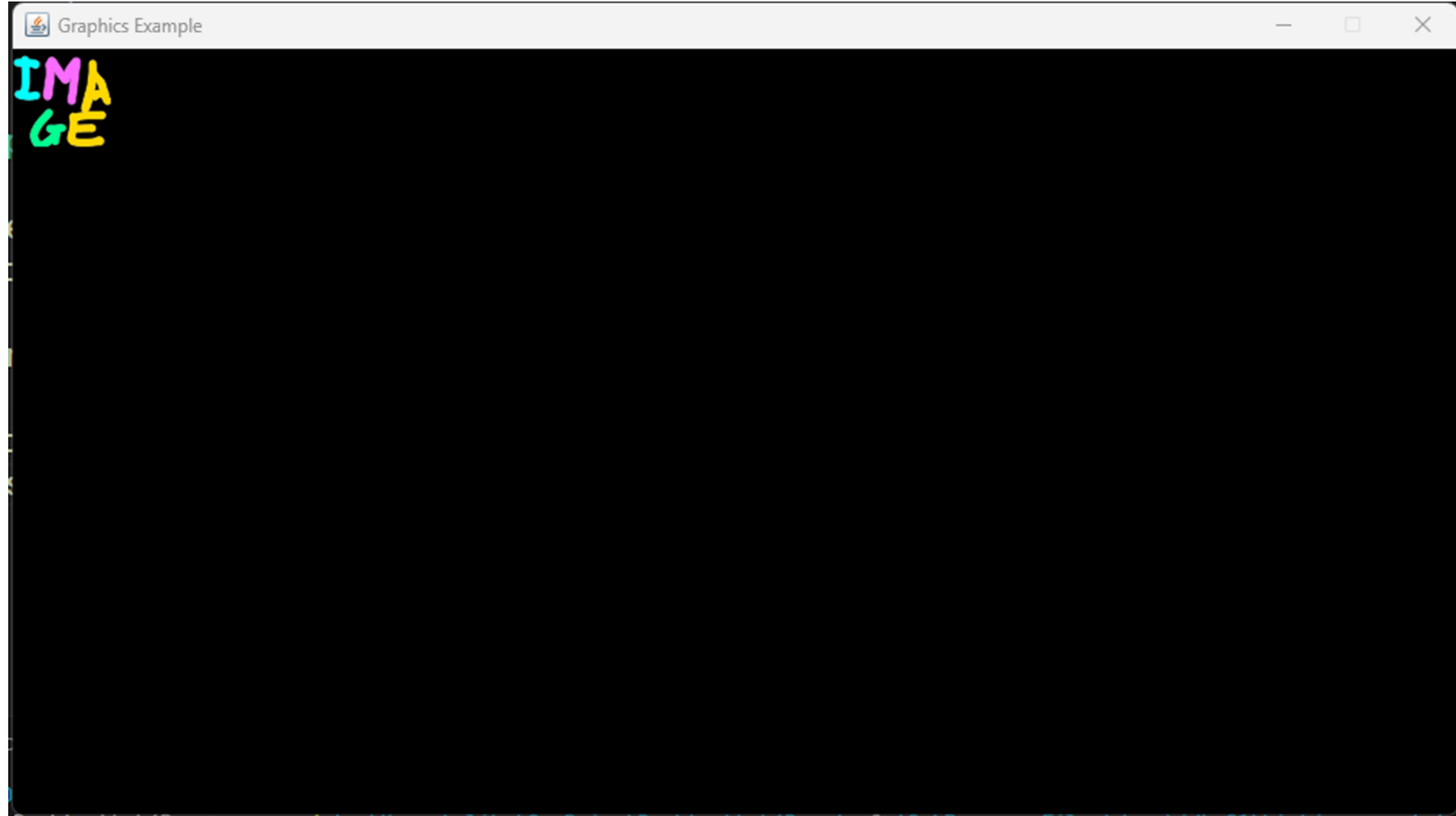
```
BufferedImage customImage = ImageIO.read(new File("CustomImage.png"));
```

Drawing images (Part 2)

- In order to draw the image, we use the `java.awt.Graphics.drawImage()` function.
- In this case, the drawing size of the image is 64x64 pixels which happens to be the actual size of the image as well. However, we can change the drawing size, and it is going to stretch the image.

```
g.setColor(Color.black);  
g.fillRect(0, 0, this.getWidth(), this.getHeight());  
  
g.drawImage(customImage, 0, 0, 64, 64, null);
```

Output



Cropping images (for sprites)

- Let's say you want to draw 4 different images. The traditional approach would be to have 4 different files. This could be wasteful in certain cases, especially when each image is relatively small.
- The better approach would be to put the 4 images into one big image and then inside the code, we can crop the one we want to use.



Cropping Images

- In the `getSubimage()` function, the first 2 parameters are the coordinates of the rectangular area that we are trying to crop.
- The other two parameters are the width and height of the rectangle to crop.

```
customImage = ImageIO.read(new File("CustomImage.png"));  
croppedImage = customImage.getSubimage(64, 0, 64, 64);
```

Drawing the cropped image

```
g.setColor(Color.black);  
g.fillRect(0, 0, this.getWidth(), this.getHeight());  
  
g.drawImage(croppedImage, 100, 100, 64, 64, null);
```



Fixing our game loop

Ideally, we want to have a fixed rate like 60 times per second to be as much as possible. In a nutshell, we want to have a fixed rate. But the render() method is not the only one that needs to be called.

We can modify our game loop to have a fixed rate.

No need to worry about the time it takes to do what it does.

```
@Override
public void run() {
    long lastTime = System.nanoTime();
    long timer = System.currentTimeMillis();
    double delta = 0;
    int updatesPerSecond = 60;

    while (true) {
        long now = System.nanoTime();
        delta += (now - lastTime) / (double) (1000000000 / updatesPerSecond);
        lastTime = now;

        while (delta >= 1) {
            tick();
            delta--;
        }

        render();

        if (System.currentTimeMillis() - timer > 1000)
            timer += 1000;
    }
}
```

Printing out the results

- If we print the frames per second (how many times `render()` is called) and the updates per second (how many times `tick()` is called), we can see that our game loop is working.
- Now we want to make sure that we don't perform game logic in the `render()` function because that function is being called unpredictable number of times per second.
- That's we should perform game logic (e.g. player movement) in the `tick()` function because it is being called at a fixed stable rate of 60 times per second.

PROBLEMS	OUTPUT	DEBUG CONSOLE	<u>TERMINAL</u>	PORTS
			Updates per second: 60, Frames per second: 1396	
			Updates per second: 59, Frames per second: 2354	
			Updates per second: 61, Frames per second: 2672	

Printing out the results

- If we print the results of the `tick()` function is called in the `render()` function.
- Now we can see the results of the `tick()` function in the `render()` function. The results are unpredictable.
- That's why we need to call the `tick()` function at a rate of 60 frames per second.

PROBLEMS	OUTPUT	DEBUG CONSOLE	TERMINAL	PORTS
	Updates per second: 60,	Frames per second: 1396		
	Updates per second: 59,	Frames per second: 2354		
	Updates per second: 61,	Frames per second: 2672		
	Updates per second: 60,	Frames per second: 2889		
	Updates per second: 60,	Frames per second: 2978		
	Updates per second: 60,	Frames per second: 2737		
	Updates per second: 60,	Frames per second: 2893		
	Updates per second: 60,	Frames per second: 2990		
	Updates per second: 60,	Frames per second: 2996		
	Updates per second: 60,	Frames per second: 2791		
	Updates per second: 60,	Frames per second: 2624		
	Updates per second: 60,	Frames per second: 2900		
	Updates per second: 60,	Frames per second: 3017		
	Updates per second: 60,	Frames per second: 2985		

der()

king.

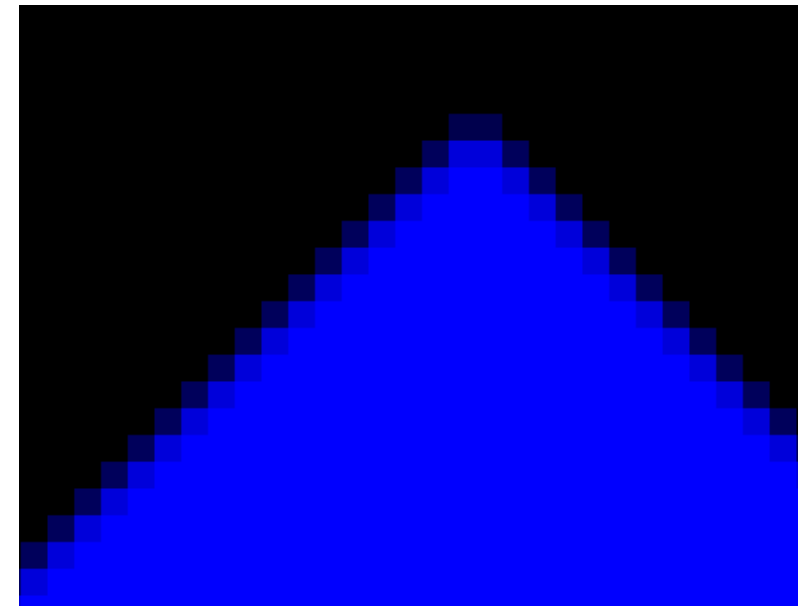
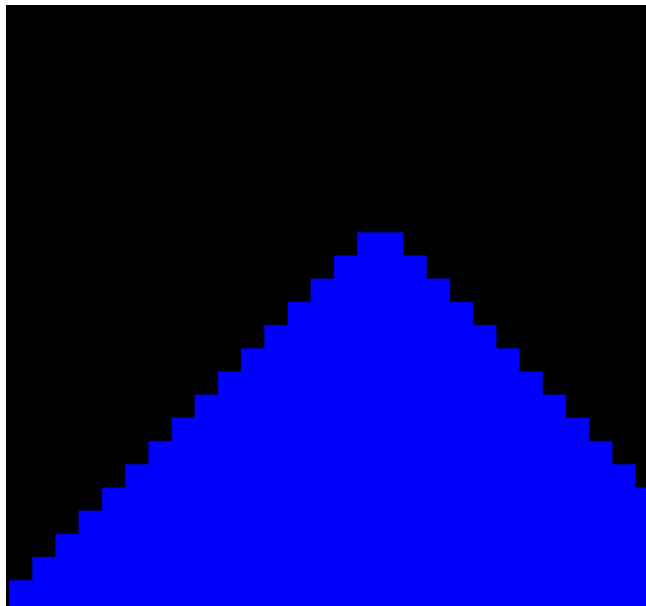
logic
g called

ment) in
stable

● Anti-aliasing



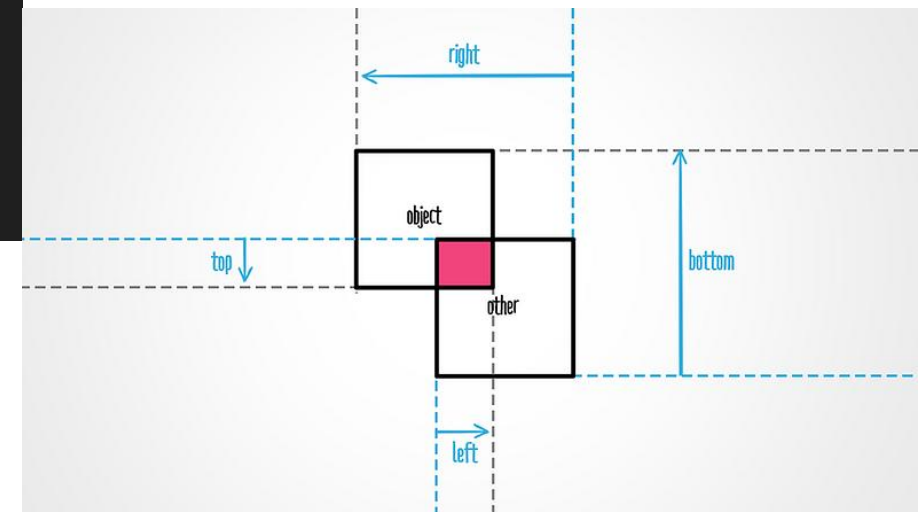
```
g2d.setRenderingHint(RenderingHints.KEY_ANTIALIASING, RenderingHints.VALUE_ANTIALIAS_ON);
```



AABB Collision Detection (Axis-Aligned Bounding Boxes)

- The `java.awt.Rectangle` class has a `.intersects(Rectangle)` method which is used to detect if 2 rectangles are overlapping/colliding with each other.

```
Rectangle rectA = new Rectangle(0, 0, 100, 100);  
Rectangle rectB = new Rectangle(200, 0, 100, 100);  
  
if(rectA.intersects(rectB))  
    System.out.println("Rect A and Rect B collided!");
```





Useful Resources





Useful Resources

- Download free sprite sheets: <https://kenney.nl/>
- Graphics guide: <https://www.geeksforgeeks.org/what-is-java-awt-graphics/>



Thank you!
Questions?

Dr. Andreas S. Maniatis

Assistant Professor

Andreas.Maniatis@UWindsor.ca

<http://www.linkedin.com/in/andreasmaniatis>



University of Windsor